Recipe Database

Stacey Gopal

INSC 384, [Fall, 2025]

Part I. Modeling

1. Needs & Business Rules

The purpose of this database is to support the storage, organization, and retrieval of recipe-related information for a structured recipe collection system. The project aims to normalize and centralize all recipe data—including ingredients, preparation steps, equipment, cuisine categories, and nutritional information—so that recipes can be easily searched, analyzed, and reused.

The database is designed to address the following needs:

- Provide a **complete, standardized way** to store recipes, including titles, servings, preparation time, cooking time, and total time.
- Maintain a **catalog of ingredients** without duplication, allowing ingredients to be reused across recipes.
- Maintain consistent **units of measure** and **preparation notes** for ingredients.
- Enable recipes to be categorized by **cuisine**, **equipment**, and **tags** (e.g., "gluten-free", "quick", "vegetarian").
- Store the **step-by-step instructions** required to prepare each recipe, ensuring correct ordering of steps.
- Store **nutritional information per serving**, when available.
- Protect data integrity by ensuring that entities such as ingredients, cuisines, and equipment exist before they are associated with recipes.
- Allow for future expansion, such as meal planning, user ratings, or scaling of ingredient quantities.

Users

The system is intended for:

- **Content creators** who input new recipes.
- **Home cooks, chefs, or researchers** who want to browse, organize, and analyze recipes.
- **Students** working on database systems who need a real-world dataset to practice modeling, normalization, and query design.

No personal identifying information is stored. The database focuses entirely on recipe content.

Business rules:

Each recipe has a unique ID and a title.
Every recipe must include at least one instruction step.
Each ingredient is stored once and can be used in many recipes.
A recipe can list multiple ingredients, each with an optional quantity, unit, and prep note.
Steps must be stored in order using a step number.
These are stored in separate lookup tables.
A recipe may have zero or many cuisines, equipment items, and tags.
Lookup values must exist before they are linked to a recipe.
Duplicate names for ingredients, cuisines, equipment, and tags are not allowed.

2. Conceptual Design and ERD

Entity Definitions

Recipe

Stores basic information about each recipe.
Key: RecipeID
Attributes: Title, Servings, PrepMinutes, CookMinutes, TotalMinutes, Source, CreatedOn, Nutrition fields
Notes: One recipe has many ingredients and many instruction steps.

Ingredient

Stores each unique ingredient.
Key: IngredientID
Attributes: IngredientName
Notes: Ingredients can appear in many recipes.

RecipeIngredient

Links recipes to the ingredients they use.
Key: RecipeIngredientID
Attributes: RecipeID, IngredientID, Quantity, UnitID, PrepNote, Optional, IngredientSortOrder
Notes: Stores ingredient details specific to a recipe.

Unit

Stores units of measurement.
Key: UnitID
Attributes: UnitName

Instruction

Stores the steps for preparing a recipe.
Key: StepID
Attributes: RecipeID, StepNumber, Instruction, TimerMinutes
Notes: Each recipe has one or more ordered steps.

Cuisine

Stores cuisine types.
Key: CuisineID
Attributes: CuisineName

RecipeCuisine

Links recipes to cuisines.
Key: (RecipeID, CuisineID) or own ID
Attributes: RecipeID, CuisineID

Equipment

Stores equipment items used in recipes.
Key: EquipmentID
Attributes: EquipmentName

RecipeEquipment

Links recipes to equipment.
Key: (RecipeID, EquipmentID) or own ID
Attributes: RecipeID, EquipmentID

Tag

Stores tags or labels for recipes.
Key: TagID
Attributes: TagName

RecipeTag

Links recipes to tags.
Key: (RecipeID, TagID) or own ID
Attributes: RecipeID, TagID

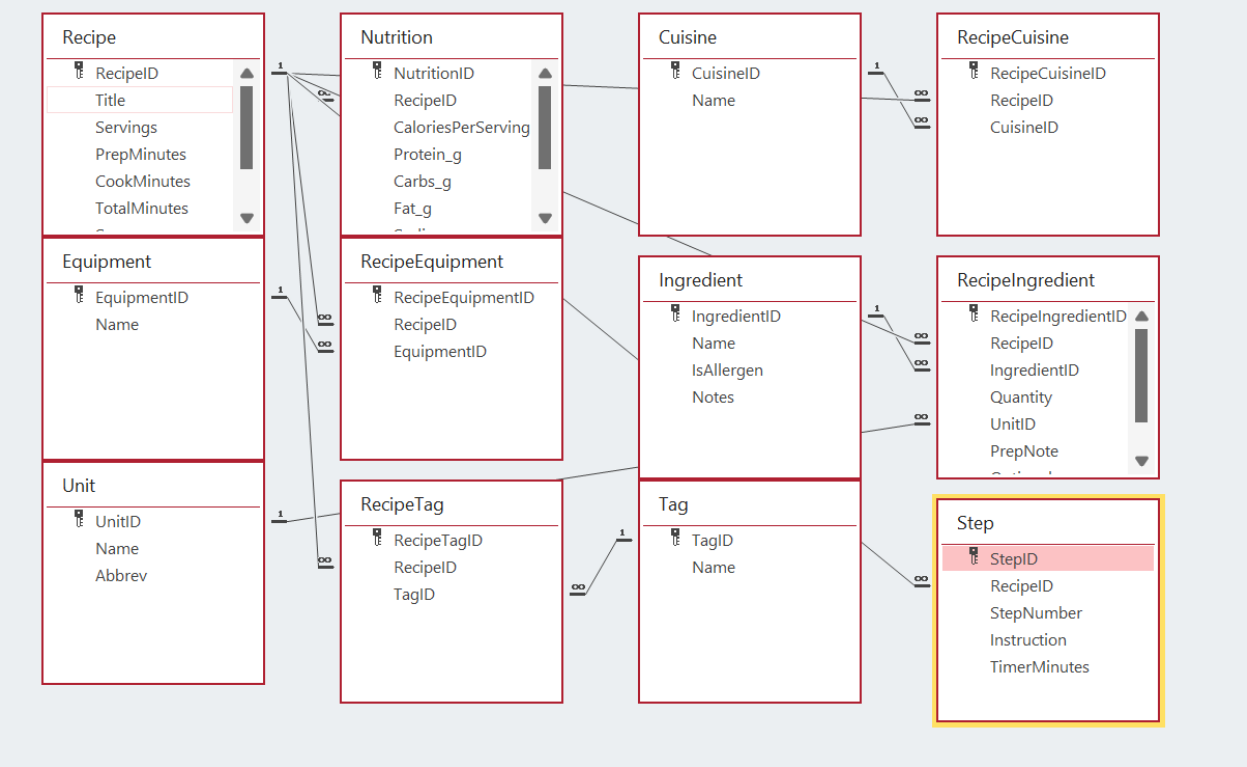


Figure 1 - ERD Diagram

The conceptual design

The conceptual design of the recipe database includes several strong entities and their relationships. Recipe is the central entity and stores the main details about each recipe. Each recipe is connected to multiple Instructions, which record the ordered steps for preparing the dish. Recipes also use multiple Ingredients, and this relationship is captured through the associative entity RecipeIngredient, which includes details such as quantity, unit, and preparation notes. Unit is a separate entity that standardizes measurement types used by ingredients.

Recipes can also be classified using several many-to-many relationships. A recipe may belong to one or more Cuisines, use various types of Equipment, or be labeled with multiple Tags; these relationships are managed through the associative entities RecipeCuisine, RecipeEquipment, and RecipeTag. Each lookup entity—Ingredient, Unit, Cuisine, Equipment, and Tag—exists independently and supports consistent, reusable values across all recipes. Together, these entities form a normalized and flexible structure for storing detailed recipe information.

3. Relational Model and Metadata

3.1 Relations/Tables

RECIPE (RecipeID, Title, Servings, PrepMinutes, CookMinutes, TotalMinutes, Source, InstructionsNote, CreatedOn, CaloriesPerServing, Protein_g, Carbs_g, Fat_g, Sodium_mg, Fiber_g, Sugar_g)
INGREDIENT (IngredientID, IngredientName)
UNIT (UnitID, UnitName)
INSTRUCTION (StepID, RecipeID, StepNumber, Instruction, TimerMinutes)
RECIPEINGREDIENT (RecipeIngredientID, RecipeID, IngredientID, Quantity, UnitID, PrepNote, Optional, IngredientSortOrder)
CUISINE (CuisineID, CuisineName)
RECIPECUISINE (RecipeID, CuisineID)
EQUIPMENT (EquipmentID, EquipmentName)
RECIPEEQUIPMENT (RecipeID, EquipmentID)
TAG (TagID, TagName)
RECIPETAG (RecipeID, TagID)

Entities / Tables (Metadata)

RECIPE

- **RecipeID (PK):** AutoNumber; unique identifier for each recipe; Required; Not Null
- **Title:** Text (200 characters); Required; Not Null
- **Servings:** Integer; Required; Not Null
- **PrepMinutes:** Integer; Optional
- **CookMinutes:** Integer; Optional
- **TotalMinutes:** Integer; Optional
- **Source:** Text (255 characters); Optional
- **InstructionsNote:** Long Text; Optional
- **CreatedOn:** Date/Time; default value = Date(); Required
- **Nutrition Fields (CaloriesPerServing, Protein_g, Carbs_g, Fat_g, Sodium_mg, Fiber_g, Sugar_g):** Number (Double); Optional

INGREDIENT

- **IngredientID (PK):** AutoNumber; Required; Not Null
- **IngredientName:** Text (100 characters); Required; Not Null; Indexed (No Duplicates)

UNIT

- **UnitID (PK):** AutoNumber; Required; Not Null
- **UnitName:** Text (50 characters); Required; Not Null; Indexed (No Duplicates)

INSTRUCTION

- **StepID (PK):** AutoNumber; Required; Not Null
- **RecipeID (FK):** Number (Long Integer); Required; Not Null
- **StepNumber:** Integer; Required; Not Null
- **Instruction:** Long Text; Required; Not Null

- **TimerMinutes:** Integer; Optional

RECIPEINGREDIENT
- **RecipeIngredientID (PK):** AutoNumber; Required; Not Null
- **RecipeID (FK):** Number (Long Integer); Required; Not Null
- **IngredientID (FK):** Number (Long Integer); Required; Not Null
- **Quantity:** Number (Double); Optional
- **UnitID (FK):** Number (Long Integer); Optional
- **PrepNote:** Text (255 characters); Optional
- **Optional:** Yes/No (Boolean); Optional
- **IngredientSortOrder:** Integer; Required; Not Null

CUISINE
- **CuisineID (PK):** AutoNumber; Required; Not Null
- **CuisineName:** Text (75 characters); Required; Not Null; Indexed (No Duplicates)

RECIPECUISINE
- **RecipeID (FK):** Number (Long Integer); Required; Not Null
- **CuisineID (FK):** Number (Long Integer); Required; Not Null
- *(Primary key may be composite or system-generated)*

EQUIPMENT
- **EquipmentID (PK):** AutoNumber; Required; Not Null
- **EquipmentName:** Text (75 characters); Required; Not Null; Indexed (No Duplicates)

RECIPEEQUIPMENT
- **RecipeID (FK):** Number (Long Integer); Required; Not Null
- **EquipmentID (FK):** Number (Long Integer); Required; Not Null
- *(Primary key may be composite or system-generated)*

TAG
- **TagID (PK):** AutoNumber; Required; Not Null
- **TagName:** Text (75 characters); Required; Not Null; Indexed (No Duplicates)

RECIPETAG
- **RecipeID (FK):** Number (Long Integer); Required; Not Null
- **TagID (FK):** Number (Long Integer); Required; Not Null
- *(Primary key may be composite or system-generated)*

4. Checking Norm Form

All tables in the design are in **1NF** and **2NF**.
All lookup and junction tables are in **3NF**.
The only exception is the RECIPE table, where TotalMinutes is a derived attribute that introduces a transitive dependency. If TotalMinutes is removed or treated conceptually as a derived field rather than a stored one, then the logical design of the database satisfies **3NF**.

Part II. Prototyping

ACCESS DBMS is used to prototype the database: Academic. accdb (Important: *Compact* database before you upload as a submission)

Phase I (Backend Structure and Queries)

1. Data Structure

See Figure 1 – ERD above

2. Queries

For information need the database can produce, including in this part, the queries. For each query (1) describe what data to output; (2) SQL statement; (3) the results as a table [(you can copy the result after run and paste the data to Word or Excel; *do not do screenshots*.]

Q1: To select all recipes and view total cooking times

```
SELECT
   Recipe.RecipeID,
   Recipe.Title,
   Recipe.Servings,
   Recipe.PrepMinutes,
   Recipe.CookMinutes,
   Recipe.TotalMinutes
FROM
   Recipe
ORDER BY
   TotalMinutes,
   Recipe.Title;
```

Results:

Query1

RecipeID	Title	Servings	PrepMinutes	CookMinutes	TotalMinutes
16	Classic Shrimp Scampi	4	0	0	15
3	Plain White Rice	4	2	20	22
2	Guyanese Paratha Roti (Oil Roti)	6	20	30	50
1	Guyanese Dhal (Bounjal method)	6	5	60	65
17	Thanksgiving Stuffing	6	0	0	80

Q2: To find all recipes using a particular ingredient

```
SELECT DISTINCT
   R.RecipeID,
   R.Title,
   I.Name
FROM
   (
      Recipe AS R
      INNER JOIN RecipeIngredient AS RI ON R.RecipeID = RI.RecipeID
   )
   INNER JOIN Ingredient AS I ON RI.IngredientID = I.IngredientID
WHERE
   I.Name LIKE "*" & [Enter ingredient name:] & "*"
ORDER BY
   I.Name,
   R.Title;
```

Results using “Parsley” as ingredient

qry_RecipeByIngredient

RecipeID	Title	Name
16	Classic Shrimp Scampi	chopped parsley

Phase II (Frontend Interactions and Main menu or Dashboard)

3. Forms or/and Reports (Optional)

frmRecipeDetails
This is the main form used to view and edit a single recipe.
Functions:

- Displays core recipe fields such as Title, Servings, Prep/Cook time, and Source.
- Hosts two subforms:
  - **fsubRecipeIngredients** (ingredient list)
  - **fsubRecipeSteps** (instruction steps)
- Allows users to edit ingredients, quantities, units, and preparation notes for the selected recipe.
- Allows editing of step-by-step instructions in the correct order.

frmSelectRecipe
This form provides a simple interface to browse or select a recipe.
Functions:

- Lists available recipes (e.g., by RecipeID or Title).
- Allows the user to choose a recipe and open it in **frmRecipeDetails**.
- Useful for navigation and quick lookup.

frmImportRecipe
Used for bringing new recipes into the database, especially if imported from external sources or spreadsheets.
Functions:

- Accepts recipe input in bulk form or from an import file.
- Runs insert logic to populate the Recipe, RecipeIngredient, and Instruction tables.
- Calls helper queries or VBA to check for existing ingredients before adding new ones.

DeleteLastImport
A utility form used for cleanup after testing or batch imports.
Functions:

- Identifies the most recently imported recipe records.
- Deletes or rolls back the last import session.
- Helps maintain data quality while developing or testing the import system.

fsubRecipeIngredients *(Subform)*
A subform embedded inside **frmRecipeDetails**.
Functions:

- Shows the ingredient list for the currently selected recipe.
- Allows editing of Quantity, Unit, PrepNote, Optional flag, and SortOrder.
- Links RecipeIngredient to Ingredient and (optionally) Unit.

fsubRecipeSteps *(Subform)*
A subform embedded inside **frmRecipeDetails**.
Functions:

- Displays the ordered steps (instructions) for the current recipe.
- Allows editing StepNumber, Instruction text, and TimerMinutes.
- Reflects the one-to-many relationship between Recipe and Instruction.

Appendix

This database was very rewarding to create on my own. Creating the form to import recipes via URL was an accomplishment for me. I certainly see where there is room for improvement, data integrity and parsing methods (i.e. Ingredient chopped parsley vs parsley (chopped)) to reduce redundancies but overall I feel very good about the completion of this task. This was a great project and would love to do it again.